

Une IA pour le jeu de Black Jack

14 décembre 2025

Vous allez programmer un jeu de Blackjack dans lequel chaque joueur sera soit un joueur humain (comme dans le rendu 1), soit l'ordinateur, auquel cas il faut définir une stratégie... gagnante si possible !

On se place dans le cas où le sabot contient un grand nombre de paquets de 52 cartes et où on mélange toutes les cartes entre 2 parties.

⇒ La probabilité de tirer une des 13 cartes possibles est donc égale à $1/13$.

1 Vocabulaire utilisé

Dans la suite, on utilise le vocabulaire suivant, pour une main de cartes :

Le **score minimum** (`minScore`) de 0 à 22 est la somme des nombres de points des cartes de la main en comptant chaque as 1 point, sauf si :

- la somme dépasse 21, auquel cas `minScore` est égal à 0, ou si
- la main est un BlackJack, auquel cas `minScore` est égal à 22.

Pour une carte, on parle de **valeur minimum** (`minValue`) de 1 à 10.

`hasAnAce` est égal à 1 si la main contient au moins un as et 0 sinon (booléen dans le programme).

Le **meilleur score** (`bestScore`) de 0 à 22 est le meilleur score. Il est égal à la somme des nombres de points des cartes de la main, en comptant un as 11 points s'il y en a au moins 1 et si cela ne fait pas dépasser 21 points.

- Le meilleur score est égal à 0 si la somme en comptant chaque as 1 point dépasse 21.
- Le meilleur score est égal à 22 si la main est un BlackJack.

On peut donc écrire :

```
bestScore = minScore + (si hasAnAce == 1 && 0 < minScore < 12 alors 10 sinon 0)
```

Le **score final** (`finalScore`) est la valeur finale du meilleur score. Le score final du croupier, s'il est non nul, est compris entre 17 et 22. Il a donc 7 valeurs possibles. Le score final du croupier correspond à un indice de colonne `y` de la matrice `dealerScoreProba` (voir plus bas).

On le représente donc par un entier `y` de 0 à 6, de la façon suivante :

- 0 représente un score nul (le croupier a dépassé 21 points),
- une valeur de `y` comprise entre 1 et 6 représente un score de $16 + y$ (compris entre $16 + 1 = 17$ et $16 + 6 = 22$).

Le tableau ci-dessous résume la correspondance entre le score du croupier et la colonne y de la matrice.

finalScore	y
0 (<i>score</i> > 21)	0
17	1
18	2
19	3
20	4
21	5
22 (Black Jack)	6

Par exemple, un score final de 21 points avec plus de 2 cartes est représenté par l'entier $y = 5$.

Quant au joueur ordinateur, il devra décider s'il « reste » ou s'il « tire ». S'il reste, son score final est son meilleur score courant. S'il tire, son score final est le meilleur score obtenu à partir de son score courant en tirant une carte.

2 Probabilités des scores finaux possibles du croupier

La matrice dealerScoreProba

`dealerScoreProba[dealerCardMinValue][y]` est la probabilité que le croupier obtienne un score final donné, à partir de la valeur minimum de sa carte visible. C'est une matrice 10×7 (10 lignes et 7 colonnes).

- **Ligne** `dealerCardMinValue` : valeur minimum de la carte visible du croupier -1 (de 0 à 9).
- **Colonne** y : représente le score final possible du croupier ($y = 0$ pour un score > 21, et $y \in [1, 6]$ pour un score $16 + y$).

Exemple

Évaluons les probabilités que le croupier obtienne les différents scores finaux possibles sachant que sa carte visible est un 6, ce qui nous permettra de remplir la ligne d'indice 5 de la matrice. Pour cela, on exécute un grand nombre de fois un tour de jeu du croupier avec un 6 comme première carte, et on comptabilise le nombre d'exécutions (fréquence) ayant abouti à chaque score final. La probabilité (empirique) d'obtenir chaque score final (sachant que la carte visible vaut 6) s'obtient en divisant la fréquence correspondante (nombre d'exécutions obtenant ce score final) par le nombre total d'exécutions.

Pour vérifier que le nombre de simulations est suffisant, vous pouvez lancer plusieurs fois votre programme de simulation et vérifier que les probabilités sont « à peu près » les mêmes : lancez 10 fois la fonction `computeDealerScoreProba` et, avec `checkSameProba`, vérifiez que la première matrice est égale aux 9 autres à 0,01 près, c'est-à-dire que chaque élément (i, j) de la première matrice est égal à chaque élément (i, j) des 9 autres matrices à 0,01 près. Si ce n'est pas le cas, vous devez augmenter le nombre de simulations.

Vous pouvez aussi vérifier que la somme des éléments d'une ligne est bien égale à 1, et que les probabilités d'obtenir un Black Jack (dernière colonne) sont celles que l'on peut calculer à la main.

3 Espérances de gain du joueur ordinateur

Pour décider s'il reste ou s'il tire, le joueur ordinateur calcule et compare son espérance de gain s'il tire une carte avec son espérance de gain s'il n'en tire pas.

3.1 Qu'est-ce que le gain ?

Le gain (positif ou négatif) est égal au produit de la mise par un coefficient (positif ou négatif). Comme l'objectif est de comparer des espérances de gains, on peut considérer que la mise est égale à 1 et que le gain est un coefficient :

- -1 si le joueur perd contre le croupier.
- 0 en cas d'égalité.
- $1,5$ si le joueur gagne (sans Black Jack).

On ne considère pas le cas où le joueur gagne contre le croupier avec un Black Jack puisqu'il n'y a pas de choix à effectuer dans ce cas.

On définit une fonction $\text{gain}(x, y)$ qui retourne le gain du joueur pour un score final x et un score final du croupier représenté par y .

3.2 Qu'est-ce que l'espérance du gain ?

C'est la moyenne des gains possibles, pondérée par leurs probabilités. Le calcul se fait en 4 étapes :

1. Identifier tous les cas possibles (scores finaux du joueur et du croupier).
2. Évaluer le gain dans chaque cas.
3. Évaluer la probabilité de chaque cas.
4. Calculer la somme, sur l'ensemble des cas, des produits : gain $\times$ probabilité.

3.3 La matrice `gainExpectedIfStands`

C'est une matrice 22×10 : `gainExpectedIfStands[bestScore][dealerCardMinValue]` est l'espérance de gain si le joueur s'arrête (*stands*).

- **Ligne** `bestScore` : meilleur score courant du joueur (0 à 21).
- **Colonne** `dealerCardMinValue` : valeur minimum de la carte visible du croupier -1 (0 à 9).

Exemple

Pour calculer l'espérance `gainExpectedIfStands[18][7]` (le joueur ne tire plus de carte, son meilleur score est 18, et le croupier montre un 8), on suit les 4 étapes ci-dessus :

1. Comme le joueur ne tire plus de carte, son score final est 18. Celui du croupier a 7 valeurs possibles. Il y a donc 7 cas possibles, représentés par les valeurs 0 à 6 de y . On va effectuer les calculs suivants pour $y = 3$, c'est-à-dire un score final du croupier de 19 points.
2. $\text{gain}(18, 3) = -1$ puisque le joueur a 18 points et le croupier 19.
3. Quelle est la probabilité que $y = 3$ (score final du croupier égal à 19) ? Comme la carte visible du croupier est un 8, cette probabilité est égale à `dealerScoreProba[7][3]`.

4. On somme finalement sur les 7 scores finaux possibles y du croupier :

$$\sum_{y=0}^6 \text{gain}(18, y) \times \text{dealerScoreProba}[7][y]$$

Vous pouvez vérifier que les éléments de la première ligne (indice 0) sont bien ceux que l'on peut facilement calculer à la main.

3.4 Le tableau `gainExpectedIfDraws`

`gainExpectedIfDraws [minScore] [hasAnAce] [dealerCardMinValue]` est l'espérance de gain si le joueur tire une carte (*draws*). C'est un tableau $21 \times 2 \times 10$.

- **Indice** `minScore` : score minimum courant du joueur (1 à 20, indice 0 non utilisé).
- **Indice** `hasAnAce` : 0 ou 1.
- **Indice** `dealerCardMinValue` : valeur minimum de la carte du croupier -1 (0 à 9).

Cette espérance est la moyenne des espérances de gain après avoir tiré une des 13 cartes possibles. Pour chaque carte tirée, on obtient un nouveau meilleur score, et l'espérance devient celle de s'arrêter avec ce nouveau score.

Exemple

Calculons `gainExpectedIfDraws [5] [1] [0]`, c'est-à-dire l'espérance de gain du joueur si son score minimum courant est 5 avant de tirer une carte, s'il a au moins un as, et si la carte visible du croupier est un as. Suivons les 4 étapes :

1. Comme le joueur tire une carte parmi 13 cartes possibles, et qu'il y a toujours 7 scores possibles pour le croupier, il faudrait normalement envisager 7×13 cas possibles. On peut néanmoins en rester à 13 cas (13 cartes possibles pour le joueur) en s'appuyant sur `gainExpectedIfStands`. Montrons le calcul pour le tirage d'un 4.
2. Après le tirage d'un 4, le meilleur score du joueur est 19 et il ne tire plus de carte. Comme la carte visible du croupier est un as, on sait que l'espérance de gain dans ce cas est égale à `gainExpectedIfStands [19] [0]`.
3. La probabilité de tirer un 4 est de $1/13$.
4. On somme finalement sur les 13 cartes possibles du joueur :

$$\frac{1}{13} \sum_{\text{rank}=1}^{13} \text{gainExpectedIfStands}[\text{theBestScore}(\text{minScore}, \text{hasAnAce}, \text{rank})][\text{dealerCardMinValue}]$$

On définit une fonction `theBestScore(minScore, hasAnAce, cardRank)` qui retourne le meilleur score d'un joueur. Ce score est défini par `minScore` et `hasAnAce` après avoir tiré la carte d'identifiant `cardRank` (de 1 à 13). Par exemple, `theBestScore(5, 1, 4) = 19`.

Noter que le tirage d'une carte peut rendre le score supérieur ou égal à 21. C'est pourquoi les lignes d'indices 0 et 21 de la matrice `gainExpectedIfStands` sont nécessaires. Par contre, dans le tableau `gainExpectedIfDraws`, la valeur 0 de l'indice `minScore` n'est pas utilisée car le joueur ne tire pas de carte s'il a dépassé 21 points.

Noter aussi que les espérances de gain du tableau `gainExpectedIfDraws` sont celles du joueur s'il tire une *unique* carte, puisque l'on considère qu'il arrête de tirer après avoir tiré une carte.

En fait, le joueur tire des cartes tant que son espérance de gain s'il tire une carte unique est supérieure (ou égale) à celle s'il ne tire pas de carte.

Suggestion : pour vous aider à déboguer votre programme, vous pouvez vérifier que tous les éléments des 2 tableaux d'espérance de gain sont compris entre -1 et 1,5!

4 Travail à effectuer

Vous complèterez la classe IA (cf. squelette ci-dessous) ayant un attribut d'instance qui est un entier de 0 à 9 égal à la valeur minimum de la carte visible du croupier – 1.

La classe possède aussi 2 attributs de classe qui sont les 2 tableaux d'espérances de gain. Ces tableaux seront générés au début du jeu. Le joueur a un attribut qui est une instance de la classe IA. Cet attribut est construit par le joueur ordinateur dès qu'il connaît la carte visible du croupier, et l'aide à faire ses choix.

Le sujet présentant l'ensemble des classes du programme objet précisera comment répondre aux 3 questions suivantes. En particulier, un menu permettra de choisir de jouer normalement (à peu près comme dans le rendu 1) ou de répondre aux questions des sections 4.2 et 4.3.

4.1 Quels choix ? Comment utiliser ces tableaux ?

A vous de voir comment utiliser ces tableaux pour :

- choisir de tirer une carte ou non,
- choisir de doubler sa mise ou non,
- choisir d'abandonner ou non.

Pour l'assurance, il y a aussi un calcul d'espérance de gain, mais un calcul simple pour lequel les tableaux précédents ne vous seront pas d'un grand secours.

4.2 La stratégie de l'IA est-elle gagnante ?

Pour le savoir, programmer un jeu entre le croupier et l'ordinateur avec un solde initial de 1000€ et une mise de 1€ à chaque partie. Le jeu s'arrête si le solde atteint 0€ (défaite), 2000€ (« victoire »), ou après 10 000 parties. Affichez les statistiques après chaque partie : le nombre de parties effectuées, le solde du joueur, les nombres de fois où le joueur a gagné, perdu, fait égalité, a abandonné. Les affichages des déroulements des parties sont supprimés (prévoir un moyen de les garder ou les enlever en modifiant un seul booléen).

La stratégie est efficace si le jeu s'arrête quand le joueur a au moins 2000€. Elle est d'autant plus efficace que le nombre de parties au bout duquel il atteint 2000€ est petit.

4.3 La stratégie de l'IA est-elle gagnante pour le « vrai » jeu ?

Adapter les règles de gain et d'assurance aux règles réelles et refaire la simulation.

- Gain normal : 2 fois la mise (gain net de 1).
- Gain avec Black Jack : 2,5 fois la mise (gain net de 1,5).
- Assurance : prime = mise/2, gain = 3 fois la prime.

5 Squelette de la classe IA

```
1 public class IA {
2
3     // attributs
4     private int dealerCardMinValue; // de 0 a 9
5
6     private static double[][] gainExpectedIfStands = new double[22][10];
7     private static double[][][] gainExpectedIfDraws = new double
8     [21][2][10];
9
10    // constructeur
11    // Pre-requis : 1 <= aCardMinValue <= 10
12    public IA(int aCardMinValue) {
13
14    }
15
16    // methodes (toutes publiques pour les tests)
17
18    /**
19     * Pre-requis : m1 et m2 ont memes dimensions, et epsilon > 0
20     * Resultat : true ssi pour tout i,j, |m1[i][j] - m2[i][j]| <=
21     epsilon
22     */
23    public static boolean checkSameProba(double[][] m1, double[][] m2,
24    double epsilon) {
25
26    }
27
28    /**
29     * Pre-requis : 0 <= i <= 9
30     * Resultat : un entier de 0 a 6 representant le score final du
31     * roudi apres une simulation d'un tour de jeu
32     * du croupier, la premiere carte tiree etant de valeur minimum i
33     +1
34     */
35    public static int simulation(int i) {
36        // en attente des autres classes
37    }
38
39    /**
40     * Pre-requis : 0 <= i <= 9 et nbSimul > 0
41     * Resultat : un tableau contenant la ligne d'indice i de la matrice
42     dealerScoreProba
43     * obtenu par nbSimul simulations
44     */
45    public static double[] computeLineDealerSP(int i, int nbSimul) {
46
47    }
48
49    /**
50     * Pre-requis : nbSimul > 0
51     * Resultat : la matrice dealerScoreProba, obtenue par nbSimul
52     simulations pour chaque ligne
53     */
54    public static double[][] computeDealerScoreProba(int nbSimul) {
55
56    }
57 }
```

```

51
52 /**
53  * Pre-requis : 0 <= bestScore <= 21 et 0 <= y < 7
54  * Resultat : le gain du joueur (-1, 0 ou 1.5) si son
55  *   score final est bestScore et celui du croupier est
56  *   represente par y
57  */
58 public static double gain(int bestScore, int y) {
59
60 }
61
62 /**
63  * Action : remplit la matrice IA.gainExpectedIfStands
64  */
65 public static void computeGainExpectedIfStands() {
66     double[][] dealerScoreProba = computeDealerScoreProba(...); //
Ex: 100k simulations
67     // ...
68 }
69
70 /**
71  * Pre-requis : 1<=minScore<=20, 0<=hasAnAce<=1, 1<=rank<=13
72  * Resultat : le meilleur score obtenu en tirant la carte identifiee
73  * par ranksur le score defini par minScore et hasAnAce
74  */
75 public static int theBestScore(int minScore, int hasAnAce, int rank)
{
76     Card c = new Card(rank); // Suppose une classe Card existante
77     // ...
78 }
79
80 /**
81  * Action : remplit le tableau IA.gainExpectedIfDraws
82  * Cette methode appelle d'abord gainExpectedIfStands
83  * (pour les tests automatiques).
84  */
85 public static void computeGainExpectedIfDraws() {
86     computeGainExpectedIfStands();
87     // ...
88 }
89
90 /**
91  * Action : remplit les deux tableaux (a appeler au debut du jeu)
92  */
93 public static void computeIAarrays() {
94     computeGainExpectedIfDraws();
95 }
96
97 // --- Methodes de decision de l'IA ---
98
99 /**
100  * Resultat : true si le joueur choisit de tirer et false sinon
101  */
102 public boolean chooseToDraw(int minScore, boolean hasAnAce, int
bestScore) {
103
104 }
105

```

```

106  /**
107   * Resultat : true si le joueur choisit de doubler la mise
108   *           et false sinon
109   */
110  public boolean chooseDoubleBet(int minScore, boolean hasAnAce, int
bestScore) {
111
112  }
113
114  /**
115   * Resultat : true si le joueur choisit d'abandonner et false sinon
116   */
117  public boolean chooseToSurrender(int minScore, boolean hasAnAce, int
bestScore) {
118
119  }
120
121  /**
122   * Pre-requis : la carte visible du croupier est un as
123   * Resultat : true si le joueur choisit de s'assurer et false sinon
124   * Mettre en commentaire ici la justification de votre resultat
125   * (calcul d'esperance simple) ==>
126   */
127  public boolean chooseInsurance() {
128
129  }
130
131 } // end class IA

```

Listing 1 – Squelette de la classe IA